Lab 2 — Reflection Report

dbt & DuckDB

EL ABDI Ibrahim
OUANZOUGUI Abdelhak

École Centrale Casablanca

Data Engineering — February 2026

Table of Contents

1. Brief Description of Final Architecture

The Lab 2 pipeline is a **dbt + DuckDB** analytics architecture that ingests raw Google Play Store data — app metadata and user reviews — and transforms it into a Kimball-style dimensional star schema suitable for analytical querying. The pipeline replaces the ad-hoc Python scripts from Lab 1 with a modular, testable, and fully reproducible framework.

Architecture Layers

The pipeline follows a classic **three-layer medallion architecture**, progressing data from raw sources through staging views into analytics-ready mart tables:

Layer	Purpose	Models
Sources	Raw JSON/JSONL files read via DuckDB's `read_json_auto()`	`apps_metadata`, `apps_reviews`
Staging	Renaming, casting, type standardisation, and null filtering	`stg_playstore_apps`, `stg_playstore_reviews`
Dimensions	Conformed dimensions for the star schema	`dim_apps`, `dim_apps_scd`, `dim_developers`, `dim_categories`, `dim_date`
Facts	Central fact table with incremental loading	`fact_reviews`
Snapshots	SCD Type 2 historical tracking	`apps_snapshot`

Source Configuration

Sources are defined in `models/staging/sources.yml`. DuckDB reads the raw files directly using the `external_location` meta property, eliminating any separate ingestion step:

```
sources:
- name: raw_data
schema: main
tables:
- name: apps_metadata
meta:
external_location: "read_json_auto('data/raw/apps_metadata.json')"
- name: apps_reviews
meta:
external_location: "read_json_auto('data/raw/apps_reviews.jsonl')"
```

Materialisation Strategy

Materialisation conventions are enforced at the folder level in `dbt_project.yml`. Staging models are materialised as **views** (lightweight, always reflecting the latest raw data), while mart models are materialised as **tables** (persisted for fast BI queries). The `fact_reviews` model overrides this default with an **incremental** materialisation (detailed in Section 2.1).

```
models:
lab2_google_play:
staging:
+materialized: view # lightweight, always fresh
marts:
+materialized: table # persisted for performance
```

DAG (Model Dependency Graph)

The full dependency graph flows from raw sources through staging into dimensions and finally into the central fact table. dbt resolves this DAG automatically via `ref()` references, guaranteeing correct execution order on every run. Figure 1 below shows the lineage graph as rendered by `dbt docs generate`.

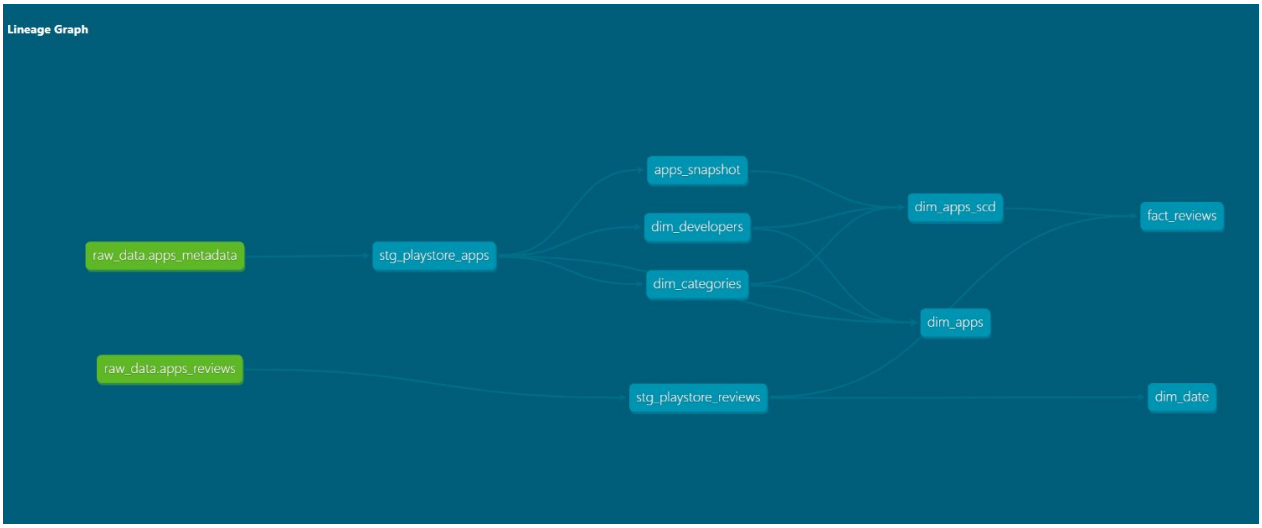

Figure 1 — dbt Lineage Graph showing the star-schema layout with fact_reviews at the far right, fed by staging models and dimension tables. Green nodes are raw sources, teal nodes are models and snapshots.

2. Implementation Details

2.1 Incremental Loading

Problem

The original `fact_reviews` model was materialised as a full `table`, meaning dbt would **drop and fully rebuild** the entire fact table on every run. For a reviews pipeline where new batches arrive daily, reprocessing millions of existing rows is unnecessary and wasteful. As the dataset grows, this approach becomes increasingly expensive in both time and compute.

Implementation

The model was converted to an **incremental materialisation** with two key additions. First, the configuration block declares `materialized='incremental'` and `unique_key='review_id'`. The `unique_key` ensures that if a row with the same `review_id` already exists, dbt will update it rather than creating a duplicate — guaranteeing idempotent behaviour.

Second, the Jinja conditional block `is_incremental()` is used inside the reviews CTE to filter only rows whose date is strictly greater than the maximum `date_key` already present in the target table. On the very first run, this filter is skipped and the full dataset is loaded. On subsequent runs, only genuinely new records are processed.

```
-- models/marts/facts/fact_reviews.sql
{{ config(
materialized = 'incremental',
unique_key = 'review_id'
) }}

with reviews as (
select * from {{ ref('stg_playstore_reviews') }}
{% if is_incremental() %}
where cast(strftime(review_timestamp, '%Y%m%d') as integer)
> (select coalesce(max(date_key), 0) from {{ this }})
{% endif %}
),
apps as (
select * from {{ ref('dim_apps_scd') }}
)

select
reviews.review_id,
apps.app_key,
apps.developer_key,
cast(strftime(reviews.review_timestamp, '%Y%m%d') as integer) as date_key,
reviews.rating,
reviews.thumbs_up_count,
reviews.review_text,
reviews.review_version
from reviews
inner join apps on reviews.app_id = apps.app_id
and reviews.review_timestamp >= apps.dbt_valid_from
and (apps.dbt_valid_to is null or reviews.review_timestamp < apps.dbt_valid_to)
where apps.app_key is not null
and date_key is not null
```

Testing the Incremental Load

To verify correct incremental behaviour, the following three-step test was performed:

- **Initial run:** `dbt run --select fact_reviews` built the full table from scratch.
- **Append mock data:** Two new JSONL records with future dates (2027-01-01, 2027-01-02) were appended to `apps_reviews.jsonl`.
- **Incremental run:** Re-running the command processed only the new rows, completing in 0.91 seconds.
- **Idempotency check:** A third run (without new data) resulted in PASS=1, no new rows inserted — confirming idempotent behaviour.

Figure 2 below shows the terminal output of the incremental run. dbt detected 11 models, 1 snapshot, and 32 data tests in the project, but only ran the single incremental `fact_reviews` model. The run completed in **0.91 seconds** with PASS=1, WARN=0, ERROR=0, confirming that only new records were processed without a full table rebuild.

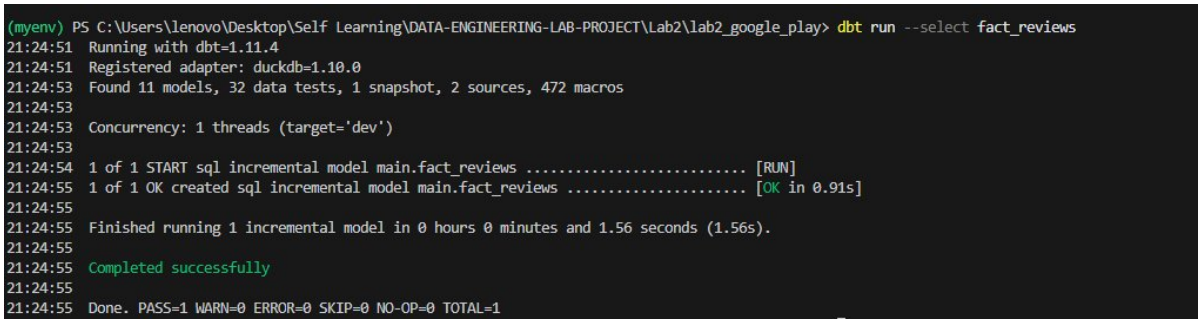

Figure 2 — Terminal output of dbt run --select fact_reviews showing incremental processing. The model completed in 0.91s with PASS=1, confirming no full rebuild occurred.

2.2 Slowly Changing Dimensions — SCD Type 2

Problem

The original `dim_apps` model overwrites app attributes on every run. If an app changes its category (e.g., from "Productivity" to "Education"), the old value is lost permanently. This makes historical analysis impossible — we cannot answer questions like "What category was this app in when this review was written?"

Implementation

SCD Type 2 was implemented using dbt's native **snapshot** functionality combined with a downstream dimension model. The snapshot definition uses the `strategy='check'` approach because the raw data does not have a reliable `updated_at` timestamp. On each snapshot run, dbt compares the listed `check_cols` — if any value differs from the stored version, it closes the old row (sets `dbt_valid_to`) and inserts a new version with updated attributes.

Snapshot Definition — `snapshots/apps_snapshot.sql`:

```
{% snapshot apps_snapshot %}
{{ config(
target_schema='snapshots',
unique_key='app_id',
strategy='check',
check_cols=['app_name', 'genre_name', 'price', 'is_paid',
'installs', 'average_rating', 'ratings_count']
) }}
select * from {{ ref('stg_playstore_apps') }}
{% endsnapshot %}
```

A downstream model `dim_apps_scd` reads from `apps_snapshot` and enriches the output with developer and category foreign keys, plus a derived `is_current` boolean flag (`dbt_valid_to is null`) for convenient filtering:

```
-- models/marts/dimensions/dim_apps_scd.sql
with apps as (select * from {{ ref('apps_snapshot') }}),
developers as (select * from {{ ref('dim_developers') }}),
categories as (select * from {{ ref('dim_categories') }})

select
row_number() over (order by apps.app_name, apps.dbt_valid_from)
as app_key,
apps.app_id, apps.app_name,
developers.developer_key, categories.category_key,
apps.price, apps.is_paid, apps.installs,
apps.average_rating as catalog_rating, apps.ratings_count,
apps.dbt_valid_from, apps.dbt_valid_to,
case when apps.dbt_valid_to is null then true else false end
as is_current
from apps
left join developers
on apps.developer_name = developers.developer_name
left join categories
on apps.genre_name = categories.category_name
```

Temporal Join in fact_reviews

The fact table joins the SCD dimension using a date-range overlap, ensuring every review links to the **exact version of the app** that was active when the review was written:

```
inner join apps on reviews.app_id = apps.app_id
and reviews.review_timestamp >= apps.dbt_valid_from
and (apps.dbt_valid_to is null
or reviews.review_timestamp < apps.dbt_valid_to)
```

Testing the SCD

To verify SCD Type 2 behaviour, the genre of "Google NotebookLM" was changed from `Productivity` to `Productivity Modified` in the raw `apps_metadata.json`. After re-running `dbt snapshot`, dbt detected the change: the old version was closed (assigned `dbt_valid_to`) and a new current version was created — confirming correct SCD Type 2 behaviour. The query result below shows both versions:

app_id	genre_name	dbt_valid_from	dbt_valid_to	is_current
com.google...tailwind	Productivity	2026-02-22 20:52:40	2026-02-22 21:01:11	false
com.google...tailwind	Productivity Modified	2026-02-22 21:01:11	NULL	true

Figure 3 — SCD Type 2 verification query result. The old version was closed and a new current version was created, confirming the historical record is fully preserved.

2.3 Data Quality & Testing

dbt's testing framework allows data quality expectations to be declared **declaratively** in YAML schema files. Tests run automatically via `dbt test` and fail the pipeline if any contract is violated. A total of **28 data tests** were defined across all layers, covering uniqueness, non-null constraints, referential integrity, and value range validation.

Staging Layer Tests

The staging layer enforces primary key integrity and basic data quality constraints:

```
# models/staging/schema.yml
models:
- name: stg_playstore_apps
columns:
- name: app_id
tests: [unique, not_null]
- name: app_name
tests: [not_null]
- name: developer_name
tests: [not_null]

- name: stg_playstore_reviews
columns:
- name: review_id
tests: [unique, not_null]
- name: app_id
tests:
- not_null
- relationships:
to: ref('stg_playstore_apps')
field: app_id
- name: rating
tests:
- not_null
- accepted_values:
values: [1, 2, 3, 4, 5]
```

Key tests include `unique` + `not_null` on primary keys to prevent duplicates, `relationships` on `app_id` in reviews to verify referential integrity (every review references a valid app), and `accepted_values` on `rating` to ensure ratings are strictly 1–5 — catching dirty data with out-of-range values.

Dimension & Fact Layer Tests

The dimension layer verifies that all surrogate keys are unique and non-null, and that foreign key relationships between dimensions are valid (e.g., every `developer_key` in `dim_apps` exists in `dim_developers`). The fact layer enforces that every foreign key (`app_key`, `developer_key`, `date_key`) points to a valid dimension row — ensuring star-schema integrity.

```
# models/marts/facts/schema.yml
models:
- name: fact_reviews
columns:
- name: review_id
tests: [unique, not_null]
- name: app_key
tests:
- not_null
- relationships:
to: ref('dim_apps')
```

```
field: app_key
- name: developer_key
tests:
- not_null
- relationships:
to: ref('dim_developers')
field: developer_key
- name: date_key
tests:
- not_null
- relationships:
to: ref('dim_date')
field: date_key
```

Running `dbt test` executes all **28 tests** across staging, dimension, and fact layers. As shown in Figure 4, every test passes with PASS=28, WARN=0, ERROR=0 — confirming data integrity across the entire star schema. The tests completed in 4.01 seconds.

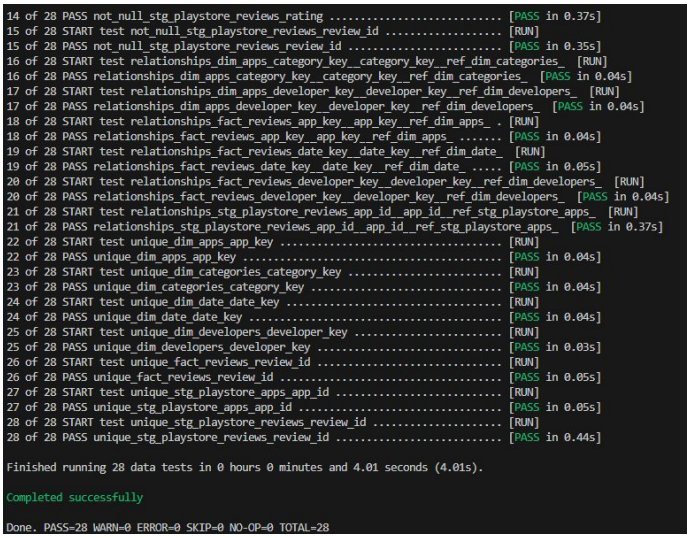

Figure 4 — Terminal output of dbt test showing all 28 tests passing. Tests cover uniqueness, not_null constraints, referential integrity (relationships), and accepted_values validation across all pipeline layers.

3. Python-Only vs dbt-Based Pipeline Comparison

The table below compares the Lab 1 Python-only pipeline with the Lab 2 dbt-based pipeline across key engineering dimensions:

Aspect	Lab 1 (Python-Only)	Lab 2 (dbt + DuckDB)
Language	Pure Python (Pandas, custom scripts)	SQL + Jinja templating
Data Storage	CSV / Pandas DataFrames	DuckDB (columnar analytical DB)
Transformation	Imperative Pandas code (manual joins, groupby)	Declarative SQL with CTEs
Testing	Manual assertions or ad-hoc print statements	28 declarative YAML-defined tests (`dbt test`)
Incremental Loading	Not built-in; requires manual diffing and state tracking	Native: `materialized='incremental'`
SCD Support	Not supported; requires custom merge logic	Native: `dbt snapshot` with check strategy
Lineage & DAG	Implicit — developer must track execution order manually	Explicit — `ref()` auto-resolves the DAG
Modularity	3 monolithic Python scripts	11+ single-purpose SQL models + YAML schemas
Idempotency	Must be manually ensured (check-before-insert)	Built-in via `unique_key`
Documentation	Separate README or inline docstrings	Integrated: `dbt docs generate`
Reproducibility	Depends on script execution order	Enforced by dbt DAG automatically

Key Takeaway

The Python-only pipeline is flexible but **fragile** — every transformation, test, and dependency must be manually coded and maintained. The dbt-based pipeline trades raw flexibility for **structure, testability, and built-in features** (incremental loading, snapshots, lineage) that would take hundreds of lines of custom Python to replicate. The most significant practical difference is reproducibility: in Lab 1, re-running a script in the wrong order could silently corrupt or duplicate data, whereas dbt's DAG enforces the correct execution order automatically.

4. Reflections

4.1 What Was the Most Fragile Part of the Pipeline?

The most fragile component is the **surrogate key generation using** `row_number()` across `dim_apps`, `dim_developers`, `dim_categories`, and `dim_apps_scd`. For example, in `dim_developers.sql`:

```
select
row_number() over (order by developer_name) as developer_key,
developer_name, developer_website, developer_email
from distinct_developers
```

This approach is fragile because **the key values are non-deterministic** — they depend entirely on the current ordering of the data. If a new developer is added whose name sorts before existing ones, every subsequent developer's key shifts. This cascades through the fact table and breaks any external system relying on stable IDs.

A second fragility point is the **join between dimension tables on natural keys** (`developer_name`, `genre_name`). If the source data has even minor inconsistencies in naming — trailing spaces, capitalisation differences — the join silently drops rows. There is no explicit test catching these "soft" join failures, making them difficult to detect.

4.2 What Was the Biggest Architectural Insight?

The biggest insight was understanding how dbt's **declarative** `ref()` **function and automatic DAG resolution** fundamentally change the way one thinks about data pipelines. In Lab 1's Python pipeline, script execution had to be carefully ordered: run ingestion first, then transformation, then serving. A mistake in ordering caused silent data corruption.

In dbt, writing `ref('stg_playstore_apps')` in `dim_apps.sql` automatically tells dbt that `dim_apps` depends on `stg_playstore_apps`. dbt resolves the entire DAG and runs models in the correct order — no matter how complex the dependency graph becomes. This makes the pipeline **self-documenting** and **impossible to execute out of order**.

The second major insight was how **schema tests replace manual validation**. In Lab 1, checking that ratings were between 1–5 required custom Python assertions that were easy to forget. In dbt, declaring `accepted_values: [1, 2, 3, 4, 5]` in the YAML schema makes the test permanent, documented, and automatically executed on every `dbt test` run. The separation of staging from marts further reinforced this: when raw column names differed from expectations, only the staging model needed updating — dimension and fact models were completely unaffected.

4.3 What Is One Design Decision You Would Change?

I would replace the `row_number()` surrogate keys with **deterministic hash-based keys**. The current approach:

```
-- Current (fragile)
row_number() over (order by developer_name) as developer_key
```

Would be replaced with:

```
-- Improved (stable)
md5(cast(developer_name as varchar)) as developer_key
```

This produces a **stable, reproducible key** that does not change when new rows are inserted or when execution order varies. It eliminates the entire class of bugs where downstream fact tables reference stale surrogate keys after a dimension rebuild. Ideally, the `dbt_utils.generate_surrogate_key()` macro would be used, which handles null values safely and follows community best practices.

Additionally, I would add a dedicated `schema.yml` entry for `dim_apps_scd` with proper tests — currently, the SCD dimension lacks dedicated schema tests, meaning it is not validated with the same rigour as other dimensions.